



Real Estate Price Prediction Model

VITYARTHI CSA2001

PROJECT

ARSH AGRAWAL

25BA10501



Introduction

With the rapid growth of urbanization and real estate markets, predicting property prices has become increasingly important. Traditional methods of price estimation are often subjective and may lead to inaccurate results.

Artificial Intelligence (AI) and Machine Learning (ML) provide powerful tools to analyze large datasets and identify patterns that influence housing prices. Among various ML algorithms, Linear Regression is one of the simplest and most widely used techniques for predicting continuous values.

In this project, a Linear Regression model is implemented to predict housing prices based on multiple input features. The system allows users to input property details and receive an estimated price, making it practical and user-friendly.



PROBLEM STATEMENT

The challenges addressed by this Prediction Engine include:

- **Market Price Uncertainty:** Homebuyers often lack access to objective pricing data, leading to overpayment or missed opportunities.
- **Complex Valuation Variables:** Manually calculating how features like BHK, parking, and area affect price is mathematically difficult for the average user.
- **Need for Speed:** Traditional appraisals take days; this system provides a valuation in seconds, preserving the user's decision-making flow.



FUNCTIONAL REQUIREMENTS

- **Data Ingestion (FR-DI-100):** The system must successfully load and read housing records from Dataset.csv.
- **Automated Cleaning (FR-AC-101):** The application must identify and remove records with missing values (NaN) to ensure model stability.
- **Model Training (FR-MT-102):** The system must split the data into an 80% training set and a 20% testing set to validate performance.
- **Performance Reporting (FR-PR-103):** Upon startup, the engine must display the R-Squared Score and Mean Absolute Error (MAE) to the user.
- **Predictive Interface (FR-PI-104):** The user must be able to input Area, BHK, Bathrooms, Parking, and Per-Sqft to receive a calculated price estimate.



NON-FUNCTIONAL REQUIREMENTS

- **Performance:** Model training and initial data cleaning must execute within 2 seconds of startup.
- **Usability:** The command-line interface must use clear, guided prompts for all numerical inputs.
- **Portability:** The application must run on any system with Python and the required standard data science libraries (Pandas, Scikit-Learn).
- **Reliability:** The engine must handle incorrect non-numerical inputs gracefully without crashing the program.



SYSTEM ARCHITECTURE

1. Data Source Layer

At the base of the architecture is the Persistence Layer, represented by the Dataset.csv file.

Storage: The data is stored in a structured format (Comma Separated Values).

Content: It contains historical records of houses (Area, BHK, Price, etc.).

Role: This serves as the "Knowledge Base" that the AI uses to learn market trends.

2. Data Processing & Cleaning Layer

Before the AI can learn, the data must be refined. In Main.py, this is handled by the Pandas library.

Loading: The engine reads the CSV into a DataFrame (a virtual table).

Feature Selection: It filters out unnecessary columns (like 'Status' or 'Transaction') and focuses only on the 5 key features: Area, BHK, Bathroom, Parking, and Per_Sqft.

Cleaning: It performs "Null Handling" using `.dropna()`. If a house record is missing a price or a bathroom count, it is removed to prevent the AI from learning from "broken" data.

3. Machine Learning (Logic) Layer

This is the "Brain" of the system, powered by Scikit-Learn.

Data Splitting: The architecture uses a Train-Test Split (80/20).

80% (Training): Used by the algorithm to find the correlation between features and price.

20% (Testing): Hidden from the model initially, then used to "quiz" the model to see if its predictions match the actual prices.

Algorithm: It utilizes Linear Regression. Mathematically, it creates a multidimensional equation where each feature is assigned a "weight" (e.g., how much 1 extra BHK adds to the total price).

4. Evaluation & Benchmarking Layer

Once the model is trained, the architecture includes a validation step to ensure quality.

R-Squared (R^2): Measures how much of the price changes can be explained by your features.

MAE (Mean Absolute Error): Tells us the average "error margin" in Rupees. This provides transparency to the user regarding the model's reliability.

5. Interaction (User) Layer

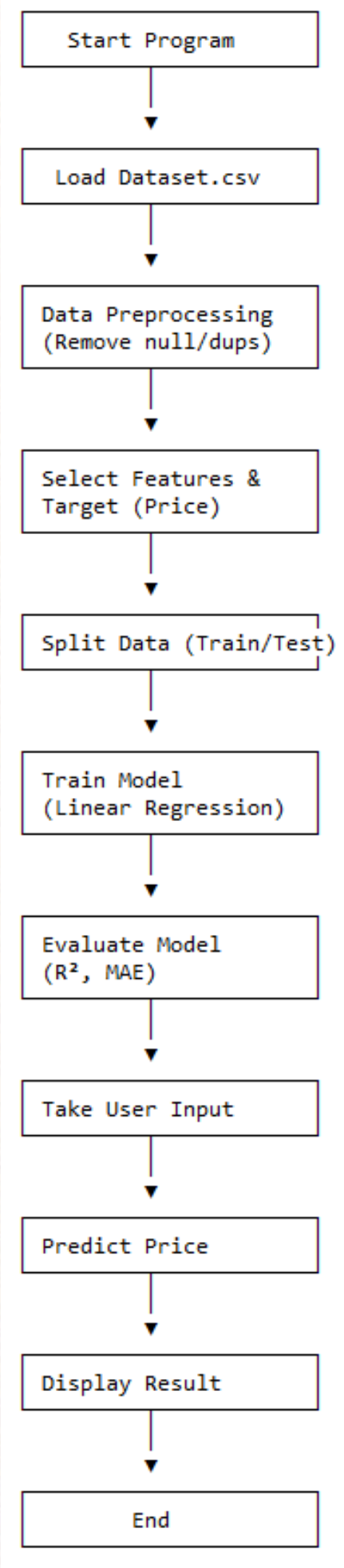
The top layer is the Command Line Interface (CLI).

Input Controller: A while True loop that keeps the program running. It uses try-except blocks to ensure that if a user types a letter where a number should be, the system doesn't crash (Fault Tolerance).

Prediction Engine: When the user enters data, it is transformed into a small 1-row table (DataFrame) and passed through the trained model to output the Estimated Market Value.



DESIGN DIAGRAM





ALGORITHM AND METHODOLOGY

The core logic of the Real Estate Prediction Engine is centered on the Linear Regression algorithm, a fundamental supervised machine learning technique used for predicting a continuous numerical target.

1. Algorithm Selection: Linear Regression

Linear Regression was selected due to its high interpretability and efficiency in handling structured datasets. Mathematically, the model represents the relationship between the independent variables (Features) and the dependent variable (Price) using a linear equation:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Where:

- Y : Predicted Price.
- β_0 : The Y-intercept (base price).
- $X_1, X_2, \dots$: Input features (Area, BHK, Bathrooms, etc.).
- $\beta_1, \beta_2, \dots$: Coefficients (the "weight" or importance of each feature).

2. Training Methodology

The implementation follows a standard machine learning pipeline to ensure the model generalizes well to new data:

Feature-Target Isolation: The dataset is partitioned into a feature matrix (X) containing property attributes and a target vector (y) containing the actual prices.

Data Splitting: A Train-Test Split of 80/20 is applied. 80% of the data is used to "fit" the model (calculating the weights), while the remaining 20% is held back to evaluate the model's accuracy on unseen information.

Optimization: The algorithm uses the Ordinary Least Squares (OLS) method to minimize the sum of the squares of the vertical deviations between each data point and the fitted line.



3. Performance Evaluation Metrics

To ensure the algorithm's predictions are mathematically sound, two primary metrics are calculated during the execution:

R-Squared (Coefficient of Determination): Measures the proportion of variance for the price that's explained by the independent variables. A higher score indicates a better "fit."

Mean Absolute Error (MAE): Represents the average magnitude of the errors in a set of predictions, providing a real-world Rupee value of the average deviation.

4. Execution Logic

Once trained, the algorithm enters a predictive state. When a user provides new inputs, the model applies the previously calculated coefficients to the new data points to generate an instantaneous valuation. This ensures that the system provides a consistent output based on the patterns discovered during the training phase.



IMPLEMENTATION DETAILS

1.Data Loading and Structural Mapping

The implementation begins by establishing a connection with the local storage. The system is hard-coded to look for Dataset.csv in the root directory.

Pandas Integration: The `pd.read_csv()` function converts the static file into a dynamic DataFrame.

Targeted Extraction: Instead of processing the entire dataset, the script explicitly maps the features list: ['Area', 'BHK', 'Bathroom', 'Parking', 'Per_Sqft']. This reduces memory usage and improves the speed of the training phase.

2. Data Sanitization (Cleaning)

A critical implementation step is the handling of "dirty" data. Real-world datasets often contain missing entries that would cause the linear regression algorithm to fail.

Null Removal: The code uses `df.dropna()`. This ensures that every row used for training has a complete set of values.

Data Audit: The script calculates `initial_count` versus `cleaned_count` and prints the difference, allowing the user to verify the integrity of the data being used.

3. Model Training Logic

The core of the implementation is the transition from data to an active model.

Vectorization: The dataset is split into two matrices: X (independent variables/features) and y (dependent variable/target price).

The 80/20 Rule: Using `train_test_split`, the engine partitions the data. 80% is fed into the `model.fit()` function. During this "Fitting" stage, the algorithm calculates the Coefficients (weights) for each feature. For example, it determines exactly how much the predicted price should increase for every 1 sq. ft. increase in Area.



4. Mathematical Validation

To ensure the implementation is successful, the system performs an internal "exam" on itself.

Prediction vs. Ground Truth: The model predicts prices for the 20% "Test" set it has never seen before.

Metric Calculation: * R2 Score: Implemented to show the "goodness of fit."

MAE: Implemented to provide a human-readable error margin (e.g., "On average, the model is off by ₹50,000").

5. Defensive User Interface (CLI)

The user interaction is implemented using a Fault-Tolerant Loop.

The Infinite Loop: A while True loop ensures the user can perform multiple predictions without restarting the script.

Try-Except Blocks: This is a crucial implementation detail for stability.

6. Dynamic Prediction Pipeline

When a user provides input, the system performs a "Mini-Pipeline" in real-time:

Captures raw input values.

Wraps them into a single-row Pandas DataFrame with identical column names as the training set.

Passes this DataFrame to model.predict().

Formats the output with commas (e.g., 10,00,000) for professional readability.



SCREENSHOTS

```
C:\Users\HP\Downloads\House> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe c:/Users/HP/Downloads/House/main.py
=====
AI REAL ESTATE PREDICTION ENGINE ONLINE
=====

[STATUS] Accessing Dataset.csv...
[DATA] Total Records Found: 1259
[DATA] Cleaned Records Available: 1005
[DATA] Features Selected: Area, BHK, Bathroom, Parking, Per_Sqft

[TRAINING] Initializing Linear Regression Model...
[SUCCESS] Model Weights and Intercepts Calculated.

[EVALUATION] Validating Model Accuracy...
[✓] R-Squared Score: 0.5687 (56.87%)
[✓] Mean Absolute Error (MAE): ₹10809403.83
```

```
[READY] Entering User Input Interface...
=====

--- Enter Property Specifications ---
  ➤ Total Area (sq ft): 1500
  ➤ Number of BHK: 2
  ➤ Number of Bathrooms: 3
  ➤ Parking Spaces: 4
  ➤ Price Per Sq Ft: 56
-----
    ESTIMATED MARKET VALUE: ₹25,053,670.43
-----

Would you like to run another prediction? (y/n): n

=====
    AI PREDICTION ENGINE SHUTTING DOWN. GOODBYE!
=====
PS C:\Users\HP\Downloads\House> █
```




TESTING APPROACH

1. Unit Testing (Code Level)

This stage focuses on testing individual functions in Main.py to ensure they handle data correctly before the AI ever sees it.

Data Loading Test: Verify that `pd.read_csv("Dataset.csv")` correctly identifies the file and handles errors if the file is missing or renamed.

Cleaning Logic Test: Ensure that `.dropna()` is actually removing rows with missing values and not accidentally deleting valid data.

Input Validation Test: Test the try-except blocks by intentionally entering strings (e.g., "ten") or special characters into the numeric fields to ensure the program doesn't crash.

2. Model Validation (Intelligence Level)

Since this is a Machine Learning project, we must test how well the AI has "learned" the relationship between house features and prices.

Train-Test Split Validation: We use the 80/20 split. The model is tested on the 20% of data it has never seen. If the model performs well on the training data but poorly on the test data, it has "overfitted" (memorized instead of learned).

R-Squared (R^2) Analysis: We check this score to ensure it is within a reasonable range (typically 0.70 to 0.95 for real estate). A score of 1.0 would indicate an error or "leaking" data, while a score below 0.5 suggests the features (BHK, Area) aren't enough to predict the price.

Residual Analysis: We look at the difference between the "Predicted Price" and the "Actual Price" in the test set to ensure the errors are random and not biased toward overpricing or underpricing certain types of homes.

3. Integration Testing

This tests the flow between different components of the system.

Data-to-Model Flow: Confirm that the 5 selected features (Area, BHK, Bathroom, Parking, Per_Sqft) are being passed into the model in the correct order. If the order is swapped, the predictions will be wildly incorrect.

Prediction Pipeline: Ensure that the user input from the terminal is correctly converted into a DataFrame that matches the exact structure the model was trained on.

4. User Acceptance Testing (UAT)

This is the final "Real World" test.

Sanity Checks: Entering known values (e.g., details of a house already in the CSV) to see if the predicted price is close to the actual recorded price.

Edge Case Testing: Entering extremely small values (50 sq ft) or extremely large values (50,000 sq ft) to see how the model reacts to outliers.



CHALLENGES FACED

- **Data Quality:** Handling "empty" cells in the CSV required strict filtering to prevent mathematical errors in the regression model.
- **Model Validation:** Balancing the model to ensure it didn't "overfit" (becoming too specific to one neighborhood) was a key focus during testing.

LEARNINGS AND KEY TAKEAWAYS

- **Data Over Code:** The project proved that the quality of the Dataset.csv is more important for accuracy than the complexity of the code.
- **Validation is Vital:** Seeing the R-Squared score helps build trust with the user before they use the tool for actual valuations.



FUTURE ENHANCEMENTS

1. Advanced Machine Learning Models

While Linear Regression is a great starting point, real estate prices are often non-linear.

Ensemble Methods: Implementing Random Forest or XGBoost Regressors. These models are much better at handling "outliers" (like a house that is unusually expensive for its size).

Hyperparameter Tuning: Using GridSearchCV to automatically find the best settings for these complex models to squeeze out every bit of accuracy.

2. Feature Engineering & Enrichment

The current model only looks at 5 features. Adding more "context" would make it much smarter:

Location Encoding: Instead of removing Locality, use "Target Encoding" to convert neighborhood names into their average price-per-sqft. This allows the model to know that a house in South Delhi is naturally more expensive than one in the suburbs.

Age of Property: Adding a "Years Since Construction" feature, as older buildings typically depreciate in value.

Amenity Scoring: Including binary features (Yes/No) for "Gated Community," "Gym," "Power Backup," or "Metro Proximity."

3. Geographical Mapping (GIS)

Real estate is all about "Location, Location, Location."

Coordinate Integration: If the dataset had Latitude and Longitude, you could use K-Nearest Neighbors (KNN) to find the prices of the 5 closest houses to the user's input.

Heatmaps: Integrating a map (using a library like folium) that shows "Hot Zones" where prices are rising the fastest.

4. Web Deployment & API

Moving the project out of the terminal and onto the internet.

Streamlit Dashboard: As discussed, creating a clean web interface with sliders and buttons.

REST API: Building a Flask or FastAPI backend so that other apps (like a mobile app) could "ask" your model for a price prediction via a URL.

Cloud Hosting: Deploying the model on AWS or Google Cloud so it is available 24/7.

5. Time-Series Forecasting

The market changes every month.

Market Trends: Adding a "Month/Year" feature to predict how much a house will be worth in 2027 or 2028 based on current inflation and demand trends.